Frontend → AI Engineer

3-Month Career Transition Roadmap

A structured week-by-week learning plan for frontend developers transitioning into backend & AI engineering roles.

Month 1	Month 2	Month 3
Python & Backend Fundamentals	Scaling & Deployment	LLMs, RAG & AI Engineering

Designed for self-learners • ~10–15 hrs/week recommended

ABOUT THIS ROADMAP

This roadmap is built specifically for **frontend developers** — people comfortable with JavaScript, HTML/CSS, and modern frameworks — who want to transition into **AI engineering**.

You will start from zero Python knowledge and progressively build up backend skills, cloud deployment expertise, and finally hands-on AI/LLM engineering capabilities over 12 weeks.

How to use this roadmap: Each week has a clear focus area with specific topics. Aim for 10–15 hours per week. Each month ends with a project that consolidates your learning. Don't skip the projects — they are your portfolio.

Prerequisites: Familiarity with any programming language (JavaScript counts), basic understanding of APIs (you've called them from the frontend), and comfort with the command line.

	Focus	Key Skills	Project
M1	Python & Backend	Python, FastAPI, PostgreSQL, Docker	Task Manager API with Auth
M2	Scale & Deploy	AWS, Redis, CI/CD, API Gateway, K8s	Deploy M1 API to cloud + pipeline
M3	AI & LLMs	OpenAI SDK, RAG, VectorDB, LangChain	Document Q&A AI API

MONTH 1

Python & Backend Fundamentals

Weeks 1–4 • Go from zero Python to a production-ready REST API

TECH STACK

Python 3.12	FastAPI
Pydantic	SQLAlchemy
PostgreSQL	Alembic
JWT / OAuth2	Docker
Pytest	Postman / Httpie

WEEK 1
Python from Scratch (for JS Devs)

Why Python for backend?

- Python vs JavaScript — syntax differences, indentation, no semicolons
- Setting up Python 3.12, pyenv, virtual environments (venv)
- pip and package management (think npm but for Python)

Python Core Syntax

- Variables, data types: str, int, float, bool, None
- Lists, tuples, dicts, sets — how they map to JS arrays/objects
- Control flow: if/elif/else, for loops, while loops, list comprehensions
- Functions: def, *args, **kwargs, default params, type hints

Python OOP & Modules

- Classes and objects — __init__, self, inheritance
- Modules and packages — importing, __init__.py
- File I/O: reading and writing files
- Error handling: try/except/finally, raising custom exceptions

Practical exercises

- Build a CLI to-do list app (add, remove, list tasks) — pure Python
- Read/write tasks to a JSON file for persistence

WEEK 2
REST APIs with FastAPI

FastAPI Fundamentals

- What is FastAPI? — async-first, auto-docs, Pydantic validation
- Creating your first GET endpoint, running with uvicorn
- Path params, query params, request body — Pydantic models
- Response models, status codes, HTTPException

Routing & Structure

- APIRouter — splitting routes into separate files (like React components)
- Dependency injection — creating reusable dependencies
- Middleware basics — logging, CORS setup
- Auto-generated docs: Swagger UI at /docs, ReDoc at /redoc

Request & Response Patterns

- CRUD pattern: Create, Read, Update, Delete endpoints
- Pydantic schema design: base, create, update, response schemas
- Nested models, optional fields, validators
- Testing with Postman or HTTPie

WEEK 3
Databases with PostgreSQL & SQLAlchemy

PostgreSQL Basics

- What is a relational DB? Tables, rows, columns, primary/foreign keys
- Install & run PostgreSQL locally (or via Docker)
- Basic SQL: SELECT, INSERT, UPDATE, DELETE, JOIN
- Indexes, constraints, transactions

SQLAlchemy ORM

- ORM concepts — models as Python classes, rows as objects
- Defining models: Column types, relationships (one-to-many, many-to-many)
- Sessions and engine setup
- CRUD with ORM — querying, filtering, updating, deleting

Migrations with Alembic

- What are migrations? (like git for your DB schema)
- alembic init, autogenerate migrations
- Upgrading and downgrading schema versions
- Connecting SQLAlchemy + Alembic to FastAPI

WEEK 4
Authentication, Docker & API Polish

Authentication

- Password hashing with bcrypt (never store plain passwords)
- JWT tokens — creation, signing, verification with python-jose
- OAuth2PasswordBearer — protecting endpoints
- User registration and login flow

Environment & Config

- Environment variables with python-dotenv (.env files)
- Pydantic Settings for typed config management
- Separating dev / staging / prod configs

Docker Basics

- What is Docker? Containers vs VMs
- Writing a Dockerfile for your FastAPI app
- docker build, docker run, port mapping
- docker-compose to spin up app + PostgreSQL together

Testing

- pytest basics — writing test functions, fixtures
- Testing FastAPI with TestClient
- Testing database operations with a test DB

■ END-OF-MONTH PROJECT

Task Management API

Build a full REST API for a task manager: user registration & login (JWT auth), CRUD for tasks (title, description, status, due date), task filtering by status, PostgreSQL database with migrations, Dockerized with docker-compose. Deliverable: GitHub repo with README, working /docs page.

MONTH 2

Scaling & Cloud Deployment

Weeks 5–8 • Take your API to production at scale

TECH STACK

AWS EC2 / ECS	AWS API Gateway
Redis	Nginx
GitHub Actions	Docker Compose
Kubernetes (basics)	Prometheus
Grafana	Terraform (intro)

WEEK 5

Cloud Fundamentals & AWS Deployment

Cloud Concepts

- IaaS vs PaaS vs SaaS — where your API fits
- AWS core services overview: EC2, S3, RDS, VPC, IAM
- Regions, availability zones, security groups
- AWS free tier setup, IAM user, CLI configuration

Deploy your FastAPI app to EC2

- Launch EC2 instance, SSH access, install dependencies
- Pull your Docker image, run container on server
- Set up Nginx as a reverse proxy in front of your app
- Configure security groups (open port 80/443)

Managed Database — AWS RDS

- Move from local PostgreSQL to RDS (managed Postgres)
- Connection strings, SSL mode, VPC networking
- Database backups and snapshots

WEEK 6
API Gateway, Rate Limiting & Caching

AWS API Gateway

- What is an API Gateway? — single entry point for all clients
- Create HTTP API in API Gateway, route to your EC2/ECS
- Request/response transformation, CORS at gateway level
- Usage plans and API keys

Rate Limiting & Throttling

- Why rate limit? Preventing abuse, protecting resources
- AWS API Gateway throttling settings (per-route, per-key)
- Implementing rate limiting in FastAPI with slowapi + Redis

Caching with Redis

- Redis data structures: strings, hashes, lists, sets
- Install Redis locally and on AWS (ElastiCache)
- Caching expensive DB queries in FastAPI with redis-py
- Cache invalidation strategies — TTL, manual bust

WEEK 7
CI/CD Pipelines & Infrastructure as Code

GitHub Actions CI/CD

- What is CI/CD? Automate test → build → deploy
- Writing your first workflow YAML — triggers, jobs, steps
- Run pytest, build Docker image, push to Docker Hub / ECR
- Deploy to EC2 via SSH action on push to main

Container Orchestration Intro

- Why Kubernetes? Scaling containers beyond single server
- Core K8s concepts: Pods, Deployments, Services, Ingress
- Run K8s locally with minikube or kind
- Deploy your app on minikube — deployment.yaml, service.yaml

Infrastructure as Code with Terraform

- What is IaC? Reproducible infrastructure
- Terraform basics: providers, resources, variables, outputs
- Provision EC2 + security group with Terraform
- terraform init, plan, apply, destroy

WEEK 8

Monitoring, Logging & Horizontal Scaling

Application Monitoring

- Structured logging with Python logging module and JSON format
- AWS CloudWatch — log groups, metric alarms
- Prometheus metrics endpoint in FastAPI (prometheus_fastapi_instrumentator)
- Grafana dashboard basics — visualising request rate, latency, errors

Health Checks & Reliability

- Health check endpoint (/health) — what to check
- AWS ELB (Load Balancer) — distribute traffic across instances
- Auto Scaling Groups — scale EC2 based on CPU/request metrics
- Zero-downtime deployment strategies: rolling, blue-green

Performance Optimisation

- Database query optimisation — EXPLAIN ANALYZE, N+1 problem
- Connection pooling with SQLAlchemy and asyncpg
- Async endpoints in FastAPI — when and how to use async/await
- Background tasks with FastAPI BackgroundTasks and Celery intro

■ END-OF-MONTH PROJECT

Production Deployment with CI/CD

Deploy the Month 1 Task API to AWS: Set up API Gateway as entry point, add Redis caching for task list queries, create a GitHub Actions pipeline that runs tests → builds Docker image → deploys to EC2 automatically on merge. Add Prometheus metrics and a basic Grafana dashboard. Deliverable: Live URL, CI/CD pipeline, architecture diagram.

MONTH 3

LLMs, RAG & AI Engineering

Weeks 9–12 • Build production-grade AI applications

TECH STACK

OpenAI SDK	Anthropic SDK
LangChain	LlamaIndex
Pinecone / pgvector	tiktoken
sentence-transformers	Pydantic AI
Weights & Biases	Streamlit (demo UI)

WEEK 9
LLM APIs & Prompt Engineering

LLM Fundamentals

- What are LLMs? Tokens, context window, temperature explained simply
- OpenAI API quickstart — chat completions endpoint
- Anthropic API — messages API, system prompts
- Comparing models: GPT-4o vs Claude 3.5 — when to use which

API Integration in FastAPI

- Streaming responses — Server-Sent Events (SSE) with FastAPI
- Handling API rate limits and retries with tenacity
- Async LLM calls — asyncio with openai AsyncClient
- Cost estimation — token counting with tiktoken

Prompt Engineering

- System prompts vs user prompts — role separation
- Few-shot prompting — giving examples in the prompt
- Chain-of-thought prompting for reasoning tasks
- Output formatting — asking for JSON, using response_format
- Prompt injection risks and basic mitigations

WEEK 10
Embeddings & Vector Databases

Embeddings Explained

- What are embeddings? Turning text into numbers that capture meaning
- OpenAI text-embedding-3-small — generate embeddings via API
- Cosine similarity — measuring how similar two texts are
- Practical demo: find similar documents with numpy

Vector Databases

- Why vector DBs? Fast approximate nearest-neighbour search
- Pinecone quickstart — create index, upsert vectors, query
- pgvector — add vector search directly to your PostgreSQL DB
- Chunking strategies: fixed-size, sentence, semantic chunking

Document Ingestion Pipeline

- Loading PDFs and text files — PyPDF2, python-docx
- Splitting documents into chunks — RecursiveCharacterTextSplitter
- Embedding chunks and storing in vector DB with metadata
- Building a simple semantic search endpoint

WEEK 11
RAG — Retrieval Augmented Generation

RAG Architecture

- What is RAG? Why it beats fine-tuning for most use cases
- RAG pipeline: Query → Embed → Retrieve → Augment → Generate
- Context window management — fitting retrieved docs in prompt
- Metadata filtering — retrieve only relevant document sections

Building RAG with LangChain / LlamaIndex

- LangChain LCEL — chains, runnables, retriever interface
- ConversationalRetrievalChain — multi-turn RAG with memory
- LlamaIndex VectorStoreIndex — simpler RAG for documents
- Hybrid search: combining semantic + keyword (BM25) search

Hallucination Control

- Model parameters: temperature (0 = deterministic), top_p, max_tokens
- Grounding prompts: 'only answer from the context below, say I don't know if unsure'
- Citation in responses — ask LLM to quote source chunks
- Re-ranking retrieved results with a cross-encoder model

WEEK 12
Evals, Observability & Production AI

Evaluating LLM Applications

- Why evals matter — you can't improve what you don't measure
- Automated evals: RAGAS framework — faithfulness, answer relevancy, context recall
- LLM-as-judge pattern — use Claude/GPT to score another LLM's output
- Building a simple eval dataset from your RAG use case

AI Observability

- Logging LLM calls — inputs, outputs, latency, token usage
- LangSmith / Langfuse — tracing LLM chains for debugging
- Weights & Biases for experiment tracking
- Alerting on quality degradation in production

Production Patterns

- Guardrails — input/output validation, content filtering
- Caching LLM responses with Redis for identical queries
- Fallback chains — try GPT-4o, fall back to GPT-3.5 on timeout
- Multi-turn conversation management — summarising long histories
- Async job queue for long LLM tasks with Celery + Redis

■ END-OF-MONTH PROJECT

Document Q&A; AI API

Build an end-to-end AI API: users upload PDFs → documents are chunked & embedded into pgvector → users ask natural language questions → RAG retrieves relevant chunks → LLM generates a grounded, cited answer. Includes: hallucination controls (temperature=0, grounding prompt, citations), RAGAS eval script, LLM call logging, streaming responses, deployed to AWS. Deliverable: Live API, Postman collection, eval report, GitHub repo.

TIPS FOR SUCCESS

→ Build every week — reading without coding doesn't work. Minimum 1 hour of hands-on per day.

→ Use AI tools (Claude, ChatGPT) to explain errors, NOT to write code for you in Month 1–2.

→ Document your projects — a good README is the difference between a portfolio piece and dead code.

→ Join communities: FastAPI Discord, Latent Space newsletter, LangChain Discord for support.

→ By Month 3, you should be building AI features, not just calling APIs — own the full stack.